

Directed Graph Acyclicity Checker

Module: **5SENG003W Algorithms and Data Structures** — University of Westminster

Student: **Nicholas Nicolaou** | ID: **W2047568**

1. Introduction

This report describes the design and implementation of a C++ program that determines whether a directed graph is acyclic or cyclic. Given a directed graph $G = (V, E)$, the program either confirms the graph is a DAG (Directed Acyclic Graph) by outputting the full sink-elimination sequence, or identifies it as cyclic and reports an explicit cycle.

The implementation spans from three files: Graph.h (class interface), Graph.cpp (algorithm implementation), and Main.cpp (entry point). Input files begin with a vertex count on the first line, followed by one directed edge per line expressed as a pair of space-separated integers.

2. Data Structure

2.1 Representation

The graph uses an **adjacency list** structure built from three parallel arrays, each indexed by an internal vertex index $0 \dots n-1$:

- $\text{adj}[v]$ — outgoing neighbour list. Stores every vertex reachable from v in one step.
- $\text{radj}[v]$ — reverse adjacency list. Stores every vertex u such that $u \rightarrow v$ is an edge.
- $\text{outdeg}[v]$ — current out-degree. The sentinel value -1 marks a vertex that has been eliminated from the graph.

Two translation maps bridge external labels to internal indices: labelToIndex (std::map , $O(\log n)$ lookup) and indexToLabel (vector, $O(1)$ reverse lookup).

2.2 Design Rationale

The three candidate representations I evaluated:

Representation	Advantage	Disadvantage
Adjacency matrix	$O(1)$ edge lookup	$O(V^2)$ space, unusable for sparse graphs at large V
Adjacency list	$O(V+E)$ space; $O(\text{deg})$ removal via radj ; ideal for sink elimination	$O(V+E)$ edge-existence test (irrelevant here)
Edge list	Minimal memory overhead	$O(E)$ to find or remove a sink, poor asymptotic performance

The reverse adjacency list radj is the critical optimisation. When a sink v is removed, every predecessor's out-degree must be decremented. Without radj this requires scanning all E edges, with it only the true predecessors of v are visited, reducing each removal to $O(\text{deg}^{-1}(v))$. Summed over all V removals this totals $O(E)$, keeping the overall algorithm linear in $V+E$ (modulo the hasSink scan (discussed in Section 4)).

2.3 Operation Complexities

Operation	Complexity	Note
$\text{addEdge}(u, v)$	$O(1)$	Append to $\text{adj}[u]$ and $\text{radj}[v]$, increment $\text{outdeg}[u]$
$\text{hasSink}()$	$O(V)$	Linear scan; first vertex with $\text{outdeg} == 0$
$\text{removeVertex}(v)$	$O(\text{deg}^{-1}(v))$	Decrement predecessors via $\text{radj}[v]$, set $\text{outdeg}[v] = -1$

isAcyclic()	$O(V^2 + E)$	V calls to hasSink ($O(V)$ each) + all removals ($O(E)$ total)
findCycle()	$O(V + E)$	Single DFS; each vertex and edge visited at most once

3. Algorithm

3.1 Sink Elimination

The acyclicity test rests on the theorem: A finite directed graph is acyclic only if it contains a sink (a vertex of out-degree zero). This yields a straightforward algorithm:

1. If the graph is empty, return Acyclic.
2. If no sink exists, return Cyclic.
3. Remove any sink and all its incident edges. Repeat from step 1.

Correctness: Every vertex on a cycle has at least one outgoing edge, so it can never become a sink. The algorithm therefore removes only non-cycle vertices until either the graph empties (acyclic) or no more sinks can be found (cyclic). Termination is guaranteed because each step strictly reduces the vertex count.

3.2 Cycle Detection

When sink elimination terminates with vertices remaining, findCycle() performs a DFS on the residual graph. A Boolean inStack[] array tracks the current recursion path. When a back edge $v \rightarrow u$ is discovered with inStack[u] true, a cycle exists. It is reconstructed in $O(\text{cycle length})$ by tracing the parent [] array from v back to u .

3.3 Worked Example

Graph 1: 10 – 9, 9 – 37, 37 – 7, 7 – 39, 39 – 25, 25 – 3, 3 – 24, 24 – 10 (cycle of length 8 embedded within a 40-vertex graph).

- **Sink elimination:** Only ten vertices can be removed (27,14,19,36,1,38,11,15,30,5). After these are eliminated, every remaining vertex has out-degree ≥ 1 , so no sink exists. The algorithm therefore returns Cyclic.
- **DFS cycle:** A depth first searches the residual graph identifies a back edge forming the cycle: 10 – 9 – 37 – 7 – 39 – 25 – 3 – 24 – 10.

Graph 2: 24 – 19, 19 – 1, 1 – 8, 8 – 23...(40 vertices in total; no vertex participates in a directed cycle).

- **Sink elimination:** At least one sink always exists. The algorithm repeatedly removes sinks, and every removal reduces the out-degree of its predecessors. All 40 vertices are eventually eliminated.
- **Result:** Since the graph becomes empty, the algorithm returns Acyclic. The elimination order produced by the program was: 24 19 1 8 23 22 2 17 34 6 35 30 25 29 39 36 15 4 26 9 27 32 18 0 20 33 21 10 7 28 31 5 12 14 37 16 11 38 13 3

4. Performance Analysis

4.1 Theoretical Complexity

The dominant cost in isAcyclic() is the repeated linear scan in hasSink(): each call costs $O(V)$, and up to V calls are made, yielding $O(V^2)$. The removal work across all steps totals $O(E)$. The overall complexity is therefore $O(V^2 + E)$. This could be reduced to $O(V + E)$ by maintaining a queue of zero-out-degree vertices, but the simpler scan is appropriate for the sizes tested here.

4.2 Benchmark Setup

Benchmarks used the 90 provided graph files (9 sizes from $V = 40$ to $V = 10\,240$, 5 instances each for acyclic and cyclic). Edge density is approximately $2.3E$ per V (sparse). Each file was timed with clock get time at nanosecond resolution; process warm-up outliers were excluded before computing per-group averages.

4.3 Results

V	Type	E	Runs	Avg (ms)	Min (ms)	Max (ms)
40	Acyclic	74	5	13.37	12.87	13.70
40	Cyclic	76	5	13.11	12.59	14.32
80	Acyclic	170	5	12.68	12.38	13.07
80	Cyclic	171	5	11.57	10.78	12.13
160	Acyclic	355	5	11.30	10.30	12.87
160	Cyclic	354	5	12.01	10.57	13.06
320	Acyclic	732	5	14.05	12.41	15.81
320	Cyclic	719	5	21.17	13.75	34.67
640	Acyclic	1,477	5	13.47	12.64	15.46
640	Cyclic	1,466	5	12.54	11.13	14.38
1280	Acyclic	2,976	5	14.08	13.76	14.69
1280	Cyclic	2,936	5	15.31	13.41	16.56
2560	Acyclic	5,954	5	17.71	17.24	18.33
2560	Cyclic	5,888	5	16.75	14.73	18.89
5120	Acyclic	11,941	5	29.14	27.71	30.57
5120	Cyclic	11,801	5	24.94	22.36	27.18
10240	Acyclic	23,885	5	84.42	62.47	92.34
10240	Cyclic	23,601	5	78.40	68.76	86.59

4.4 Discussion

At small sizes ($V \leq 1\,280$) runtimes cluster between 11–16 ms regardless of graph type - this floor is dominated by process start-up and file I/O rather than algorithm work. Algorithmic growth becomes visible from $V = 2\,560$ onwards. The jump from $V = 5\,120$ (29 ms) to $V = 10\,240$ (84 ms) - roughly $2.9\times$ for a $2\times$ size increase - is consistent with the predicted $O(V^2)$ growth from the hasSink scan.

Cyclic graphs run faster than acyclic ones at larger sizes because the algorithm terminates as soon as no sink is found often after eliminating only a small fraction of vertices, whereas acyclic graphs require all V vertices to be removed.

5. Conclusion

The adjacency list with reverse adjacency is the right choice for this problem: it enables $O(\deg^{-1})$ sink removal rather than the $O(E)$ alternative, and its $O(V+E)$ memory usage scales well with graph size. The sink-elimination algorithm is simple, provably correct, and runs in $O(V^2+E)$ with the current linear-scan hasSink(). A queue-based variant could achieve $O(V+E)$ if larger graphs were needed. DFS cycle reconstruction adds no extra asymptotic

cost. The benchmark results on all 90 provided files confirm behaviour consistent with the theoretical analysis across all tested sizes.